

Cross-Competition Benchmark Summary: generic baseline vs full skill workflow vs after Phase B iteration vs after tree search

Generated by: docs/scripts/build_benchmark_table.py (numbers come from docs/benchmark_facts.json , narrative written by the agent). Covers the 10 playground-series competitions in this weekend's batch, comparing four score tiers per event:

- **tier1** — generic baseline: **run directly by Claude Code, without invoking the kaggle-agent skill** (competitions/run_competition.py , a generic batch script, fixed LGB+XGB+CAT blend, no EDA, no feature engineering, no per-event LLM decisions)
- **tier2** — best score of the full skill workflow (Phase A, before the Phase B self-improvement iteration commit)
- **tier3** — best final score of linear iteration (Phase A + Phase B self-improvement iteration, **excluding tree search**)
- **tier4** — best score after tree search (Phase D–G), i.e. the tree-search result recorded in Phase G-1a/G-1b stacked on top of tier3 (tree search beats or ties tier3 in 9/10 events, see the "tree search" section below)

The four tiers form one ablation comparison: tier1→tier2 isolates the value of the skill's six-stage workflow, tier2→tier3 isolates the value of self-improvement iteration and the experience library, tier3→tier4 isolates the value of the search-based design (corresponding to Aygün et al.'s tree-search thesis).

Contents

Main table (10 competitions)	1
Addendum: s3e19 same-CV-scheme diagnostic comparison (not counted in the table above, for CV-scheme-bias reference only)	2
Tree search (tier4): how much more can be squeezed out after linear iteration	3
One sentence per event: where the key difference comes from	4
Verified cross-competition recipes	5
Honest notes and comparability limitations	5
How to reproduce	7

Main table (10 competitions)

Competition	Metric	Direction	tier1 (generic)	tier2 (skill/Phase A)	tier3 (linear-iteration fir
s3e1 (California housing)	rmse	↓	0.56166	0.558768	0.557088
	roc_auc	↑	0.81624	0.832925	0.83814

Competition	Metric	Direction	tier1 (generic)	tier2 (skill/Phase A)	tier3 (linear-iteration fir
s3e3 (attrition prediction)					
s3e5 (wine quality, QWK)	quadratic_weighted_kappa	↑	0.47871	0.52687	0.56769
s3e7 (booking cancellation)	roc_auc	↑	0.89882	0.899395	0.899893
s3e9 (concrete strength)	rmse	↓	12.54287	12.073474	12.070034
s3e11 (media cost)	rmsle	↓	0.29723	0.296143	0.295648
s3e14 (blueberry yield)	mae	↓	341.40782	340.711795	340.59891
s3e16 (crab age, rounded MAE)	mae(rounded)	↓	1.35441	1.33812	1.33812
s3e19 (sales forecast, SMAPE, TimeSeriesSplit)	smape	↓	no comparable generic (see CV note below)	10.175397	10.019463
s3e20 (Rwanda CO2)	rmse	↓	28.3424 (proxy value, see note below)	22.6488	21.1487

Note: relative change is always positive in the "better" direction (minimize metrics = $(\text{tier1} - \text{tierN}) / \text{tier1} \times 100$; maximize metrics = $(\text{tierN} - \text{tier1}) / \text{tier1} \times 100$). For s3e19, because tier1's CV scheme differs from tier2/tier3/tier4's and is not comparable, these two columns instead report tier2→tier3 and tier2→tier4; s3e20's tier1 is a proxy value, details in the "CV scheme and comparability note" below. For s3e9, the tree search only **exactly ties** (rather than beats) the linear best, so tier4 equals tier3, an honestly recorded tie rather than an omission.

Addendum: s3e19 same-CV-scheme diagnostic comparison (not counted in the table above, for CV-scheme-bias reference only)

Comparison	Metric	Direction	generic (plain KFold)	skill (same plain KFold, diagnostic only)	relative change
s3e19-kfold-diagnostic	smape	↓	5.31891	4.281421	+19.51%

This row and the s3e19 row of the main table use **different CV schemes** and the two are not mutually comparable (see the CV note below); this Phase G-1b tree search was run only on the honest

TimeSeriesSplit track (see the tier4 in the main table's s3e19 row), and the plain-KFold diagnostic track never went through tree search, so this comparison table has no tier4 column.

Tree search (tier4): how much more can be squeezed out after linear iteration

Phase D–G ran the harness v2/v3 tree search over all 10 competitions (node space = full solution: feature set + model combination + hyperparameters, rather than linear iteration's one-item-at-a-time), and formally merged each event's best result into `experiments.json` via `log_experiment_v2` (notes all cite the source tree file `experiments_tree*.json` node id). Result: **9/10 events beat the linear-iteration final score (tier3), 1 event (s3e9) exactly ties** — the tier1→tier4 relative change further widens beyond tier1→tier3, with s3e5 (+18.59%→+19.21%) and s3e20 (+25.38%→+25.70%) showing the largest gains. Common mechanisms observed across events:

1. **Blend contribution ≠ solo score:** in several events the largest gain came from members that are "not especially strong in solo score but receive the largest blend weight" (s3e1's CEILING mixed classifier, s3e14 mega-blend's EXPL_CATDEEP/ EXPL_REGDEEP, s3e19's CALSUBSET).
2. **Capacity push beyond the existing search boundary:** when Optuna's own tuning result happens to land on the search-space boundary (s3e11's CatBoost depth=10 upper limit), the boundary itself is the next direction to try — pushing to depth=12 was that event's single largest source of gain.
3. **Seed bagging on already-tuned models is context-dependent:** "seed bagging on an Optuna-tuned target," generally thought to be possibly neutral, is instead a decisive lever under a context like TimeSeriesSplit (s3e19) where folds are non-interchangeable and model variance is inherently large.
4. **Structure-dominated terrains still have tree-search room:** even in a structure-dominated competition where GBDT has entirely zero weight (s3e20), "jointly moving" multiple already-hand-tuned structural hyperparameters (JOINT lineage) can still find a composite optimum that the one-item-at-a-time Phase-B protocol structurally cannot reach.
5. **An exact tie is also an honest output:** s3e9's tree search exactly ties (rather than beats) the linear best (12.070034) on data whose noise is already saturated — an independent confirmation that the event's data noise ceiling has been reached by linear iteration, not a search failure.

Honest limitations (tier4, applying to 9/10 events; s3e16 is now the sole exception): the tier4 scores of these 9 events are all **OOF-only tree-search products** — no test predictions were produced, none submitted to Kaggle, and the corresponding facts.json records have no `submission` field. s3e19's tier4 additionally inherits tier2/tier3's existing fold-5 double-dip caveat, and adds its own scale-parameter/seed-selection OOF-fitting caveat (see the main-table note above and each competition's REPORT.md). s3e20's tier4 explicitly **does not adopt** node #28's sibling BLEND node score 21.0332 (recorded in STATUS.md as a low-confidence marginal finding under 3-fold CV), but instead adopts the robust pure-structure node 21.0589. s3e16's tier4 (1.33563) continues tier2/tier3's existing "rounded metric" — and is this event's second raw/rounded inversion case: that node's raw OOF MAE (1.35712) is actually worse

than the linear champion's raw 1.35589, but its rounded OOF MAE is better, again proving that this event's decision metric must be the rounded score. **s3e16's tier4 is now no longer OOF-only**: this node's test predictions have been rebuilt and were actually submitted to Kaggle on 2026-07-06, obtaining Public 1.34315 / Private 1.33859, both better than tier2/tier3's corresponding linear-champion submissions (Public 1.34356 / Private 1.34075) — it is the only competition among the 10 in this table with real-LB anchors for both tier2 **and** tier4 (see "Honest notes and comparability limitations" below).

One sentence per event: where the key difference comes from

- **s3e1**: geographic nearest-distance/KNN density features (not target encoding) + Optuna fold-proxy-tuned LGB added to the pool + seed bagging; coarse-grained geographic target encoding is itself redundant for trees that can already split coordinates, with no gain.
- **s3e3**: first removing class-imbalance weighting and tightening LGB regularization gave the largest single gain; later Optuna directly targeting full-CV ROC-AUC found a solution shallower and, counterintuitively, more lightly regularized than hand-tuning; CatBoost's native categorical handling narrowed the gap but weight search still ruled it out.
- **s3e5**: replacing naive rounding with OptimizedRounder (tuning cut points on OOF QWK) is the largest lever; the second wave of gains came from setting Optuna's objective directly to post-processed QWK for tuning LGB/CatBoost, rather than tuning an RMSE proxy first and applying post-processing after.
- **s3e7**: first pruning the cyclical encoding of date columns unrelated to the target let the scored solution overtake the baseline; adding an Optuna fold-0-proxy-tuned LGB to the pool (not replacing) was the single largest round gain, whereas repeating the same tuning recipe for a second model instead dragged down blend diversity.
- **s3e9**: regularization and feature engineering must be applied together to combat the label-noise ceiling caused by duplicate rows; once tuning and denoising themselves plateaued, seed bagging was the only lever that kept working in Phase B.
- **s3e11**: fold-safe group (store) target encoding is the largest single gain; adding an Optuna-tuned CatBoost to the pool + seed bagging stacked a small gain, while adding same-group feature aggregation then reverted (the signal was already fully consumed by the existing TE).
- **s3e14**: first pruning near-perfectly collinear features let the scored solution overtake the baseline; "adding" an Optuna fold-proxy-tuned LGB to the pool rather than replacing the original model brought the largest gain, while Ridge stacking and nested isotonic calibration were both counterexamples (losing to the simplex-grid blend).
- **s3e16**: target is integer, so rounding post-processing is the largest lever (and the rounded CV score has an extremely small gap to the real Kaggle LB, highly credible); Phase B verified the recipe boundary — Optuna tuning + seed bagging improved raw OOF but worsened rounded OOF for two consecutive rounds, stopping per protocol, tier2=tier3.

- **s3e19**: test is a future period, so TimeSeriesSplit rather than random KFold is the only honest CV (see the diagnostic comparison above; the same config looks much more optimistic under the wrong CV); after log1p target + calendar features established the Phase A foundation, Phase B's largest gain came from an LGB with Optuna proxy tuning on the "last fold," plus two rounds of seed bagging; proportional decomposition (a structured method) instead lost to GBDT in this context where the total level itself is hard to extrapolate.
- **s3e20**: discovering the target is "nearly constant across years" let pure location-week historical mean beat all GBDT (all three GBDT weight searches went to zero) as the single largest lever; Phase B then applied three-stage denoising to this historical mean (empirical-Bayes shrinkage, downweighting anomalous years, neighbor-week smoothing) for additional gain, and residual diagnostics confirmed the structural signal is already saturated — not even the residual layer can squeeze out more predictability.

Verified cross-competition recipes

1. **Optuna (fold-proxy or direct final-metric optimization) → add to the pool (not replacing the original member) → seed bagging**: consistently verified effective on s3e1, s3e3, s3e7, s3e9, s3e11, s3e14, s3e19 — currently the most durable recipe with the largest sample size; the known systematic boundary is s3e16 — when the target must be rounded, the same recipe's raw OOF gain may fail to cross the discrete rounding boundary, and decisions must use the rounded rather than raw score.
2. **Three treasures of metric-aware post-processing**: for classification/count integer targets, round directly (s3e16); for ordinal targets, pair with OptimizedRounder tuning cut points on the OOF metric (s3e5, far better than naive rounding); for discrete-grid targets, snap to the nearest training-set observation (s3e14). All three win stably over "no post-processing" or "clip only," and all must be verified with the metric itself (not a proxy loss) to confirm the gain holds.
3. **The boundary between structure and GBDT**: when the target is "nearly constant across years" on some grouping dimension (s3e20), pure historical mean/target encoding can beat all GBDT and weight search consistently gives tree models 0 weight; but when the total level itself drifts year over year and cannot be extrapolated (the proportional-decomposition counterexample of s3e19), structural decomposition provides no more information than GBDT's native categorical splits. The durable criterion is not "whether structure exists in the data" but "whether the target is nearly constant along the dimension corresponding to that structure."

Honest notes and comparability limitations

- **CV-only, except s3e16 (both tier2 "and" tier4 are LB-validated)**: the other scores in this table (including the tier4 of the other 9 events) are all Out-of-Fold cross-validation scores, not submitted to the Kaggle leaderboard (mostly closed competitions or no-credential environments). s3e16 is the sole exception, and the exception has now expanded to tier4: tier2/tier3's corresponding linear-

iteration result was actually submitted on 2026-07-03 (Public 1.34356 / Private 1.34075, CV↔LB gap extremely small, CV credible); **s3e16's tier4 (tree-search node #15) has also rebuilt test predictions and was actually submitted on 2026-07-06**, obtaining Public 1.34315 / Private 1.33859, both better than tier2/tier3's submissions — no longer CV-only, this is the first external (real-leaderboard) validation this tree-search recipe has obtained, details in the "s3e16 rounded metric" item below.

- **tier4 is CV-only and unsubmitted in nearly all events (except s3e16):** 9/10 events' tier4 is the tree-search result recorded via `log_experiment_v2` in Phase G-1a/G-1b, all OOF-only search products — the `tree_search` harness did not produce test prediction files for these 9 events, the corresponding `experiments.json` notes all have no `submission` field, and must not be mistaken for submitted scores (details and per-event caveats in the "Tree search (tier4)" section above and each competition's `REPORT.md`). **s3e16's tier4 is the sole exception among these 10 events** — see the previous item and the "s3e16 rounded metric" item below.
- **s3e19's CV-scheme difference (now covering tier2, tier3, tier4):** the generic baseline (tier1, 5.31891) uses random-shuffle KFold, while the skill workflow (tier2/tier3) and tree search (tier4) all use `TimeSeriesSplit` — because the test is a strictly future period, random KFold is an interpolative, optimistic estimate, and the two are **not directly comparable** (the iron rule of CV design in `knowledge/experience.md`). The main table therefore marks s3e19's tier1 as "no comparable generic," and the relative-change columns instead report tier2→tier3 and tier2→tier4; a separate same-CV-scheme (plain KFold) diagnostic comparison is attached above (no tier4, because tree search was run only on the honest `TimeSeriesSplit` track), presented separately and not merged into the main table's relative-change computation. tier4 (9.75707) additionally carries the fold-5 double-dip and the both-scale/seed-OOF-fitted caveats, and the true expected 2022 SMAPE should be read as "clearly below 10.02" rather than the literal 9.76 (see each competition's `REPORT.md`).
- **s3e16 rounded metric (now covering tier2, tier3, tier4; both tier2 and tier4 are real-LB-validated):** tier2/tier3's 1.33812 is the OOF MAE "after rounding to integers," not the raw `blend_oof_mae` (1.35589) that `collect.py`'s generic adapter outputs directly. Phase B ran Optuna tuning + seed bagging per the verified recipe; raw OOF did improve, but rounded OOF worsened for two consecutive rounds, so tier2 and tier3 have the same value — this is a verified recipe boundary, not a skipped iteration. tier4's tree-search result (1.33563) is likewise only a winner under the rounded metric: its raw OOF MAE (1.35712) is worse than the linear champion's raw 1.35589, the second raw/rounded inversion case for this event, and the decision must be based on the rounded score. **Update (2026-07-06):** the tier4 node has rebuilt test predictions and was actually submitted to Kaggle, obtaining Public 1.34315 / Private 1.33859, both leaderboards better than tier2/tier3's corresponding submissions (2026-07-03, Public 1.34356 / Private 1.34075) — this is the first real-leaderboard validation this tree-search recipe obtained on s3e16, not merely an internal OOF comparison (corresponding to the `leaderboard` field of `experiments.json` #5; full formula in `competitions/playground-series-s3e16/REPORT.md` § 6).
- **s3e20 has no true generic baseline:** this event is a 2023 closed, old competition completed in an old session (2026-02-14) before this weekend's batch, not within this round of `run_competition.py`'s batch, with no `sub_generic_*` files generated. Its own two earliest records used a single time-based split, no CatBoost, no target encoding, and are not comparable with later Leave-One-Year-Out CV

records, so they were not mislabeled as tier1. The main table's tier1 (28.3424) is the GBDT blend under the same CV scheme before adding location-week target encoding, used as an approximate comparison rather than a strict generic-batch baseline. This event's Late Submission is closed, and tier1–tier4 all cannot actually be submitted.

- **s3e20's tier4 explicitly excludes the low-confidence GBDT-blend node:** node #28's (21.0589, adopted in this table) sibling BLEND node (21.0332, not adopted) wins only under 3-fold Leave-One-Year-Out CV and with GBDT weights fit directly to the same OOF, and STATUS.md explicitly records it as a marginal finding within CV noise, not a robust conclusion — adopting the pure-structure node for tier4 is a deliberate conservative choice, not an oversight.
- **s3e9's tier4 = tier3 (tie, not an omission):** s3e9's tree search (harness v2) exactly ties (rather than beats) the linear-iteration best 12.070034, indicating the event's data noise ceiling has been reached by linear iteration; docs/scripts/build_benchmark_table.py's `is_tree_entry()` detected no tree-search note record in s3e9's experiments.json, so tier4 mechanically equals tier3, faithfully reflecting this tie.

How to reproduce

```
uv run python3 docs/scripts/build_benchmark_table.py
uv run python3 .claude/skills/kaggle-report/assets/verify_report.py docs/benchmark_summary.md docs/benchmark_summary.md
bash .claude/skills/kaggle-report/assets/md2pdf.sh docs/benchmark_summary.md
```